CYBERSECURITY TASK -: 1 apple_pi3

We were tasked to analyze a linux executable , recover a hidden flag and investigate its security protections and weaknesses . The hidden flag in the form of gdg{...} .

1. Executive summary

The challenge requests recovery of a hidden flag from a Linux binary, a concise assessment of security protections and weaknesses, and reproducible evidence. Static analysis identifies a 32-bit little-endian x86 ELF position-independent executable. A helper function initializes an encoded 22-byte string and XORs each byte with 0x05 before printing it. Reversing that transformation recovers the candidate flag:

gdg{P1E_3xpl01ted_lol}

The main function also passes user-controlled input directly as the format argument to printf, strong static evidence of a format-string vulnerability. PIE and a non-executable stack are present; metadata suggests partial RELRO. No stack-canary symbol was found in the inspected symbol tables. Runtime impact and acceptance by the official flag checker have not been independently verified.

2. Environment & Target Information

I performed a Static analysis initially using Fedora Workstation 44 . Runtime testing was performed in an Ubuntu Server virtual machine through virt-manager on fedora as a host .

- Host OS -> Fedora Workstation 44
- Analysis VM -> Ubuntu Server
- Virtualization platform -> KVM/QEMU with vert-manager
- Guest architecture -> x86_64
- Target architecture -> 32-bit i386

Tools used -

- sha256sum
- file
- strings
- Readelf
- Objdump
- GDB

3.Static Analysis

I used file , sha256sum , strings , readelf , and objdump to examine the binary structure , embedded strings , symbols , program logic and security mitigations .

The target was identified as a 32-bit ,little endian ELF executable for the intel 80386 architecture . The executable is dynamically linked , uses /lib/ld-linux.so.2 as its ELF interpreter , and retains its symbol information .

The ELF header was examined using `readelf -h`. The binary has ELF class `ELF32`, machine type `Intel 80386`, and executable type `DYN`, indicating a position-independent executable (PIE). Its entry point is located at virtual address `0x10a0`.

The `strings` utility was used to extract readable strings embedded in the executable. These included the program banner, password prompt, incorrect-password message, and the prompt for additional user input. These strings provided initial clues about the program's intended behavior.

Symbol information was examined using `readelf` and `objdump`. The binary contains a function named `main`, located at `0x1254`, and a function named `def_nothing_important`, located at `0x11cd`.

The presence of the latter function suggested that it warranted further investigation. Disassembly revealed that it initializes a sequence of encoded bytes and applies a byte-wise XOR operation with the constant `0x05`. Decoding the embedded data revealed the flag:

`gdg{P1E_3xpl01ted_lol}`

The disassembly of `main` did not show a direct call to `def_nothing_important`, suggesting that the flag-recovery function is not part of the ordinary control flow visible in `main`.

4. Dynamic Analysis

The `apple_pie` executable was run in an Ubuntu Server virtual machine using a non-root user. The VM's network interface was disconnected during execution. The required 32-bit runtime was installed because the target is a dynamically linked 32-bit ELF executable.

The program displayed the banner `The Elusive PIE`, requested a password, and printed an incorrect-password message when supplied with `wrong`. It then prompted for additional input. Tests using ordinary inputs such as `hello` and `12345`, as well as the format-string input `%s`, repeatedly resulted in a segmentation fault after the second input was printed.

GDB was used to investigate the crash. A breakpoint was placed at the end of `main`, at its final `ret` instruction. Before executing this instruction, the stack was inspected:

(gdb) x/wx $esp

0xffffd4c4: 0x00000001

The instruction pointer was then observed after executing the `ret` instruction:

0x00000001 in ?? ()

This confirms that the function attempted to return to address `0x1`, resulting in an invalid control transfer and segmentation fault.

Dynamic analysis confirmed that the program repeatedly crashes after processing its second input. GDB demonstrated that the crash occurs when `main` executes its final `ret` instruction and transfers control to address `0x1`.

Static analysis separately identified an unsafe `printf` call that uses user-controlled input as its format string. Although this is a confirmed format-string vulnerability, the exact mechanism by which the invalid return address becomes `0x1` was not established.

The hidden flag was recovered through static analysis of the XOR-decoding function rather than through normal runtime execution.

5. Flag recovery

5.1 Decoding routine

The symbol table identifies `def_nothing_important` at virtual address 0x11cd. Its disassembly shows immediate values written into a stack buffer, followed by a loop that reads a byte, XORs it with 0x05, and writes the result back. The loop compares the index with 0x15 and branches while it is below or equal, processing indices 0–21 (22 bytes).

5.2 Decoding rule

The integer constants are stored in little-endian byte order. Each byte is decoded as `encoded_byte XOR 0x05`. XOR is reversible, and applying it to the 22 stored bytes produces:

`gdg{P1E_3xpl01ted_lol}`

The result matches the required gdg{...} shape. The flag was recovered statically without executing the binary. Acceptance by the official checker remains unconfirmed.

6. Reproducible Evidence & Proof of Concept

Proof of concept: Disassembly shows that the second user-input buffer is passed directly to `printf` without a fixed format string, demonstrating a format-string vulnerability. The helper function `def_nothing_important` applies XOR with `0x05` to embedded bytes, recovering the flag `gdg{P1E_3xpl01ted_lol}`.

During runtime testing in an isolated Ubuntu VM, the program displayed its prompts but repeatedly terminated with a segmentation fault after the second input. The exact cause of the crash was not established.

The readelf -h command displays the ELF header of the file .



The output confirms 32-bit binary , little endian , Intel 80386 architecture ,and DYN indicates it is a PIE binary allowing the OS to load it at random memory addresses for security .

The readelf -lw command displays the Program headers .

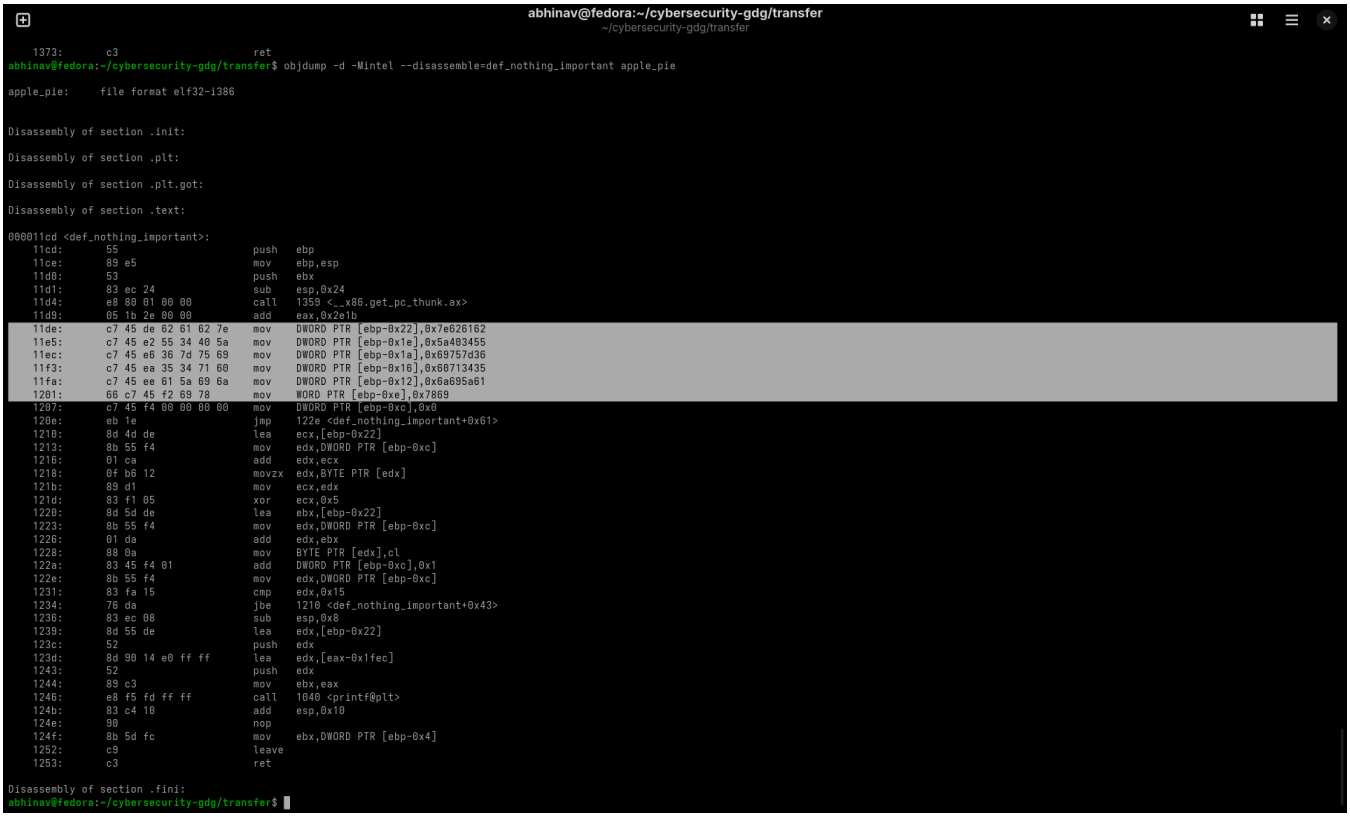
The readelf -d command displays a dynamic section of the ELF binary .

```
abhinav@fedora:~/cybersecurity-gdg/transfer$ readelf -d apple_pie

Dynamic section at offset 0x2ef0 contains 26 entries:
  Tag        Type                         Name/Value
 0x00000001 (NEEDED)                     Shared library: [libc.so.6]
 0x0000000c (INIT)                       0x1000
 0x0000000d (FINI)                       0x1360
 0x00000019 (INIT_ARRAY)                 0x3ee8
 0x0000001b (INIT_ARRAYSZ)               4 (bytes)
 0x0000001a (FINI_ARRAY)                 0x3eec
 0x0000001c (FINI_ARRAYSZ)               4 (bytes)
 0x6ffffef5 (GNU_HASH)                   0x1ec
 0x00000005 (STRTAB)                     0x2dc
 0x00000006 (SYMTAB)                     0x20c
 0x0000000a (STRSZ)                      200 (bytes)
 0x0000000b (SYMENT)                     16 (bytes)
 0x00000015 (DEBUG)                      0x0
 0x00000003 (PLTGOT)                     0x3ff4
 0x00000002 (PLTRELSZ)                   48 (bytes)
 0x00000014 (PLTREL)                     REL
 0x00000017 (JMPREL)                     0x440
 0x00000011 (REL)                        0x400
 0x00000012 (RELSZ)                      64 (bytes)
 0x00000013 (RELENT)                     8 (bytes)
 0x6ffffffb (FLAGS_1)                    Flags: PIE
 0x6ffffffe (VERNEED)                    0x3c0
 0x6fffffff (VERNEEDNUM)                 1
 0x6ffffff0 (VERSYM)                     0x3a4
 0x6ffffffa (RELCOUNT)                   3
 0x00000000 (NULL)                       0x0
abhinav@fedora:~/cybersecurity-gdg/transfer$
```

We run our objdump command on the function def_nothing_important .

Command -> objdump -d -Mintel --disassemble=def_nothing_important apple_pie



We observe some programs . These programs have hexadecimal values , so the data might have been encoded in here . Later we also have a line xor ecx, 0x5 . This tells us each byte is XORed with 0x05. So now we decode it , we know x86 uses little-endian byte order .

Decoding it further we achieve the expected output gdg{P1E_3xpl01ted_lol}

7. Control-flow note

The inspected disassembly of `main` does not show a direct call to `def_nothing_important`. The helper exists and contains the decoding routine, but the inspected main control flow does not establish that normal execution reaches it. The flag was recovered by analyzing the helper statically, not by demonstrating a runtime path through the password prompt.

8. Remediation and Mitigations

- **Fix the format-string vulnerability: Replace `printf(buffer)` with `printf("%s", buffer)` to prevent user input from being interpreted as formatting directives.**
- **Enable stack protection: Compile with `-fstack-protector-strong` to detect certain forms of stack corruption.**
- **Enable full RELRO: Use `-Wl,-z,relro,-z,now` to protect relocation data against modification.**
- **Retain PIE and NX: Keep PIE and a non-executable stack enabled to improve resistance to code-reuse and injected-code attacks.**
- **Enable compiler warnings: Use `-Wall -Wextra -Wformat=2 -Wformat-security` to help detect unsafe code.**

These changes should be validated through recompilation, static inspection, and runtime testing. The unsafe `printf` call is confirmed by disassembly, although its exact relationship to the observed segmentation fault remains unverified.

9. Conclusion

The encoded data in `def_nothing_important` decodes to gdg{P1E_3xpl01ted_lol}. The executable uses PIE and marks its stack non-executable; it also contains a GNU_RELRO segment. Dynamic metadata suggests partial RELRO, and the usual stack-canary symbol was not found. The most notable code-level weakness is passing user-controlled input directly as a `printf` format string. These are static-analysis conclusions; runtime exploitability and official flag acceptance have not been claimed.